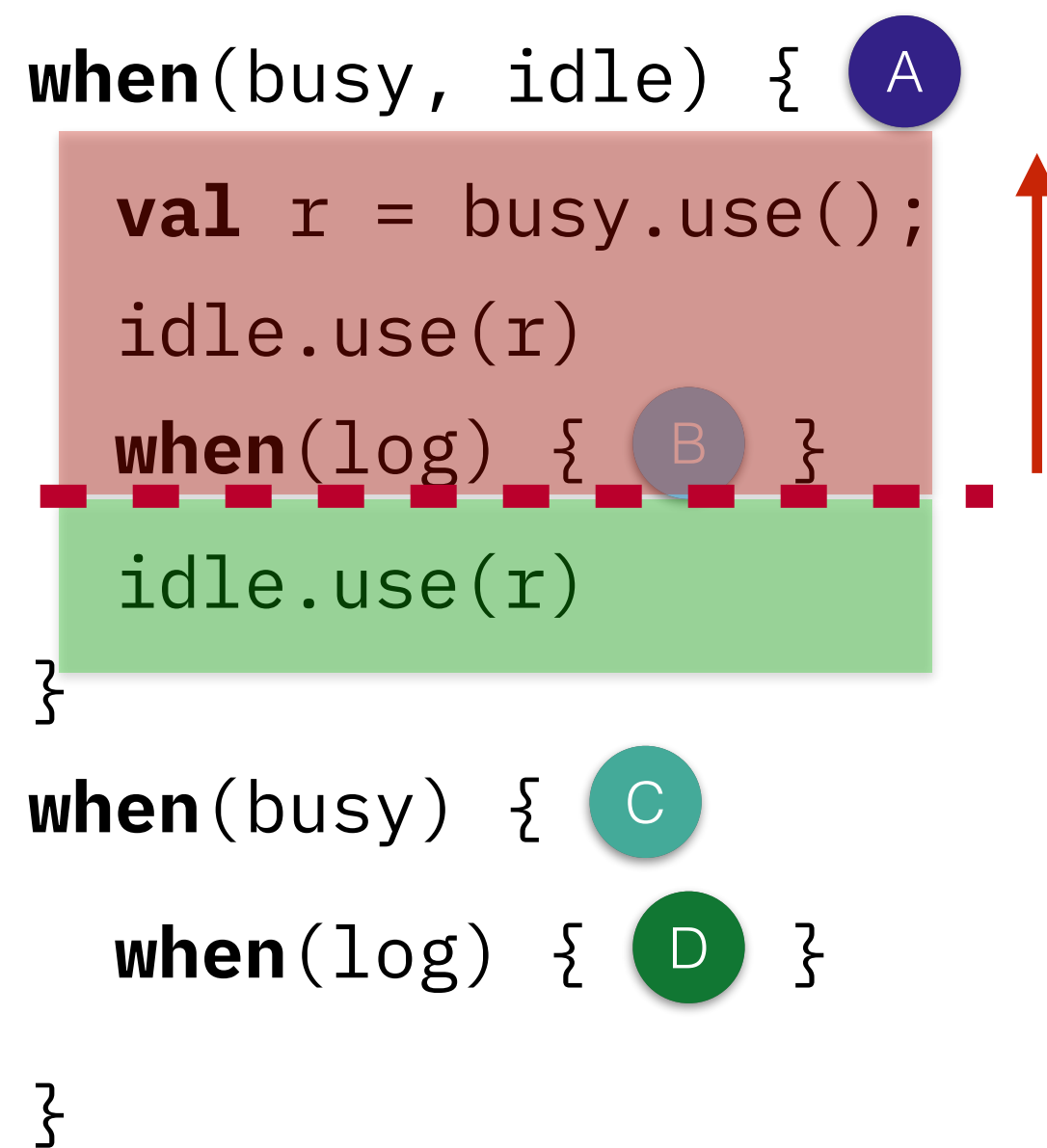


What does this get us?

We can reason about program transformations

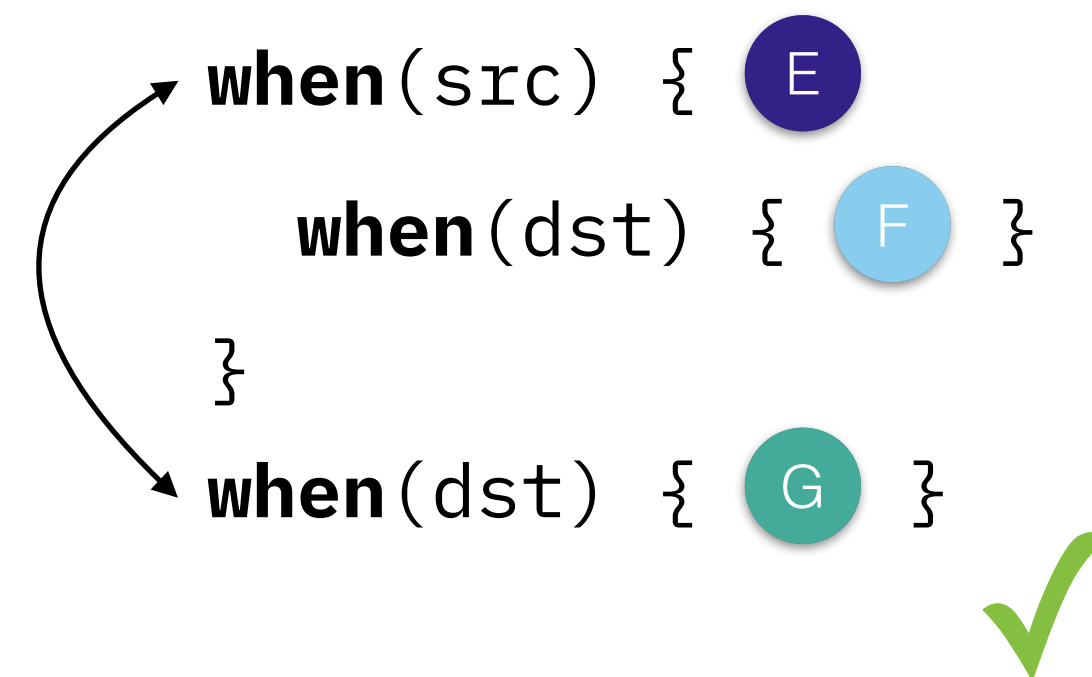
```
when(busy, idle) { A
  val r = busy.use();
  idle.use(r)
  when(log) { B }
  idle.use(r)
}
when(busy) { C
  when(log) { D }
}
```



Can we release cows early?

After the last behaviour spawn

```
when(src) { E
  when(dst) { F }
}
when(dst) { G }
```



Can we reorder behaviours?

As long as we aren't violating causality

```
when(dst) { G }
when(src) { E
  when(dst) { F }
}
```





Future and Conclusion

- Constructed an axiomatic model for Behaviour-oriented Concurrency
- Captured the intended causal order between behaviours
- Proven that our earlier work, and its derivatives, are sound with respect to this model
- Provided an implementation-independent foundation for reasoning about BoC programs
- Enabled development of a later memory model for languages that use BoC